

项目概述

1.1. 项目背景及研究内容

1.1.1 研究背景

随着大语言模型与智能体技术在企业级场景中的落地，人工智能系统正从“被动工具”向“具备自主决策与执行能力的智能体”转变。在金融风控、工业智能制造、生物医药研发、IT 运维管理等面向企业的核心场景中，智能体已展现出显著价值，包括多步骤复杂任务的自动规划与执行，如供应链优化中的动态调度、金融投研中的多源数据深度分析；跨系统、跨工具的信息整合与操作，实现异构系统的无缝协同，提升数据流转效率；多智能体协同决策与分工协作，通过任务拆解与资源调度，完成单智能体难以处理的大规模复杂任务。

然而，当前主流的智能体系统仍存在“强执行、弱学习”这一核心瓶颈：

1. 智能体在执行过程中会产生大量高价值的轨迹数据，包含决策路径、工具调用序列、环境反馈等关键信息，但这些数据通常仅用于日志记录或调试，未能转化为可复用、可演进、可审计的决策知识；

2. 智能体的能力提升主要依赖提示工程迭代、人工规则修补以及模型整体升级，这种外部驱动模式导致知识沉淀周期长、成本高，缺乏面向系统性决策经验的自动沉淀与强化机制；

3. 在企业复杂环境中，智能体的决策往往需要满足可解释（决策依据透明化）、可审计（操作全程留痕）、可复盘（结果追溯分析）、可持续优化（能力持续迭代）的要求，但现有方法难以提供系统性支撑。

因此，如何将智能体的执行过程转化为可演进的企业级知识资产，构建具备自我总结、自我强化、自我修正能力的智能体系统，成为当前智能体技术从“可用”迈向企业级“可信、可规模化”的关键问题。

本项目聚焦于企业级 Agent 知识强化的核心关键问题：如何将大量的跨用户、跨 Agent 的系统日志记录转化为可辅助 Agent 决策的知识，构建一套“可解释、可审计、可复用”的 Agent 知识的提取和演进算法。具体表现为三大技术挑战：

1. 如何将大量跨 Agent，跨用户的轨迹数据系统的组织，加工为高精度且可复用的知识：在企业级 Agent 规模化部署背景下，数据孤岛问题进一步加剧，如销售、客服、研发等部门的系统存在异构数据格式（文本日志、操作序列、交互记录等），且 Agent 轨迹数据分散在独立运行单元中。现有方案仅支持单点数据提取，缺乏跨系统语义对齐与动态知识结构化能力，导致知识复用率不足。

2. 如何有效利用大量单分散在 Agent 轨迹中的碎片化部落知识、先例知识、例外知识：企业级 Agent 运行中，隐性知识碎片化问题突出，如团队内部经验，历史成功案例，异常处理经验仅存于单 Agent 轨迹，传统 RAG 仅能检索显性知识，隐性规则挖掘率不足；碎片知识可信度参差，缺乏普适性验证机制，导致复用时错误率高。

3. 如何通过 Agent 运行数据，实现动态的知识自演进，从而提升 Agent 的规划决策能：当前 Agent 知识库多为静态更新，无法适应业务动态变化（如新政策）。无法自演进是核心瓶颈，且知识更新过程缺乏可审计性，导致企业合规风险高。

1.1.2 现有方法的局限

1. 检索增强生成：尽管 RAG^[1-3]技术在知识检索方面取得了显著进展，但在企业级知识增强 Agent 场景中存在明显局限：1) 显性知识局限：传统 RAG^[3]仅能检索预定义的显性知识库内容，无法有效挖掘和利用 Agent 轨迹中隐含的部落知识、先例知识和例外知识。在企业场景中，高价值决策经验以碎片化形式存在于 Agent 执行轨迹^[1]中，而 RAG 对此类隐性知识的挖掘率不足。2) 静态知识库缺陷：RAG 依赖预先构建的静态知识库，无法实现知识的动态演进。企业业务环境动态变化，而 RAG 缺乏实时从 Agent 运行数据中提取新知识并更新知识库的能力，导致知识时效性差。

2. Agent 记忆：当前 Agent^[4-7]记忆机制主要聚焦于短期对话上下文存储，难以支撑企业级知识资产的构建，缺乏长期知识沉淀和结构化能力。1) 在企业复杂任务场景中，需要将跨任务、跨会话的经验进行系统性整合，而当前 Memory 机制无法支持这种知识的长期存储与演进。2) 各 Agent 的记忆系统相互独立，缺乏跨 Agent 经验共享机制^[4]。企业中的 Agent 产生的轨迹数据分散在独立运行单元中，形成严重数据孤岛，无法实现知识的跨部门复用。3) Memory 机制缺乏知识提炼能力：Memory 仅存储原始交互数据，缺乏将执行轨迹转化为结构化决策知识的能力。企业级场景需要从大量轨迹中提取可复用、可演进的决策模式，而现有 Memory 机制无法自动完成这一知识提炼过程^[6]。4) 企业级应用需要完整的操作留痕和决策追溯能力，但现有 Memory 设计未考虑知识更新的可审计性^[7]，导致知识演进过程缺乏合规保障，增加了企业合规风险。

3. Agent 轨迹处理：虽然 Agent 轨迹数据^[8-10]包含丰富决策经验，但现有技术未能有效利用：1) 当前系统将 Agent 轨迹数据仅用于日志记录或调试，未能将其转化为可复用的知识资产。在企业级场景中，这些轨迹包含的决策路径、工具调用序列、环境反馈等关键信息，是构建高精度知识体系的核心素材。2) Agent 轨迹中包含大量碎片化隐性知识，如内部经验、历史成功案例、异常处理经验^[9,10]，但现有技术缺乏有效提取、验证和整合这些碎片知识的方法，导致隐性知识可信度参差，复用时错误率高。3) 当前轨迹数据处理多采用静态更新方式，无法实现知识的自动、实时演进，导致 Agent 决策能力滞后于业务发展。

1.1.3 研究内容

本项目面向企业级智能体规模化部署场景，围绕“**轨迹数据治理—知识提炼—知识组织—决策增强—持续演进**”的技术闭环，构建可解释、可复用、可持续更新的知识强化 Agent 方法体系与原型平台。研究将以统一轨迹表示为数据底座，以技能化知识沉淀与技能知识图谱为核心载体，并通过在线反馈驱动知识自演进，系统性提升 Agent 的规划决策能力与运行稳定性。项目整体研究框架如图 1 所示，具体拟从以下 5 个方面展开研究：

1) 轨迹数据治理与统一表示框架：面向跨智能体产生的异构轨迹数据，研究统一的轨迹 Schema 与语义对齐方法，将原始日志规范化为可计算、可链接的“任务—约束—计划—动作—观测—结果”结构化时序对象；同时建立轨迹完整性、一致性与可复现性等质量评估与分层采样机制，为后续知识提炼、验证与回放提供高质量数据底座。

2) 轨迹到技能的知识提炼与规格生成：针对轨迹中碎片化的部落知识、先例知识与例外处理经验，研究从长链条轨迹中自动切分可复用子过程、定位关键决策点并生成技能单元的方法；进一步研究技能聚类归并、相似度度量与跨场景迁移评估，实现技能的可复用沉淀与可解释复盘。

3) 技能知识图谱构建与知识组织：在技能沉淀基础上，研究可扩展的技能图谱(Skill Graph)表示与存储机制，刻画技能之间的依赖、替代、互斥、约束与版本关系，并将证据链与指标画像绑定到技能节点；同时研究冲突检测与一致性维护机制，支持知识的版本化管理、灰度发布与快速回滚，提升知识组织的可治理性与可维护性。

4) 知识驱动的规划决策增强：研究技能图谱驱动的约束感知技能检索与编排方法，生成可执行的 DAG 并完成可执行性检查；进一步研究多路径路由与工具替代策略选择、异常检测与自动恢复机制，使知识资产进入决策回路，在端到端任务成功率、稳定性等指标上可量化提升。

5) 在线反馈驱动的知识持续演进机制：面向业务规则变化、工具能力迭代与数据分布漂移，研究从运行轨迹与任务结果中触发的增量学习与自演进算法，建立“新增知识提案—离线验证与回归测试—在线灰度与监控—指标反馈更新”的闭环；重点解决知识更新的时效性与可靠性平衡问题，通过漂移检测、遗忘控制与失败样例回灌，实现知识与策略的持续优化，保证系统长期运行的稳态性能。

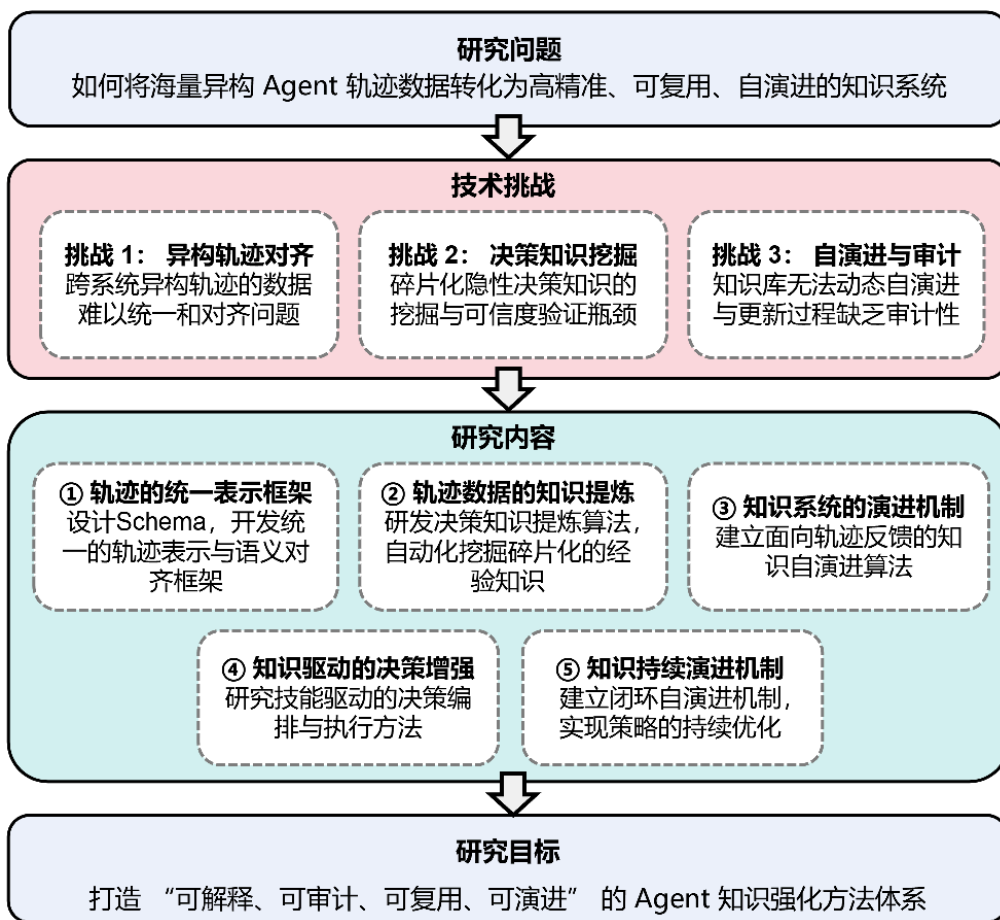


图 1: 项目整体研究框架

1.2. 项目技术方案、路线及目标

1.2.1 总体技术方案

本项目提出一种基于技能驱动的自演进知识强化 Agent 的技术方案(见图 2)。与传统“从轨迹抽规则、堆知识库”的方式不同, 本方案以可执行技能作为连接轨迹数据与决策能力的关键中间层, 将复杂任务还原为由相互依赖技能组成的工作流, 通过技能结构化、图谱化与在线反馈驱动演进, 使经验知识能够稳定沉淀并持续反哺规划决策与运行稳定性。整体技术路线按以下五个环节推进并形成闭环:

首先面向典型的智能体部署场景采集原始 Agent 运行轨迹, 或通过仿真环境回放历史任务并注入异常合成轨迹数据。然后设计统一轨迹表示 Schema, 对轨迹进行预处理与统一表示, 抽取关键事件、状态转移、异常分支与恢复动作等要素, 并对跨系统的术语与字段进行语义对齐, 形成可计算、可链接的标准化轨迹对象, 为后续知识加工提供一致的语义底座。

其次, 解析后的标准化轨迹进入技能化知识加工器。该模块面向企业场景中大量隐性部落知识、先例知识与例外知识, 不再仅生成松散的文本总结, 而是从成功/失败轨迹对比中诱导稳定的“条件—步骤—工具链—结果”模式, 将其提炼为可执行的技能单元, 并为每个技能自动生成结构化规格。同时, 引入技能可信评估机制, 通过回放重演、对照验证与一致性检验对技能的可迁移性与适用边界进行评分, 降低碎片经验复用带来的错误风险。

在此基础上, 构建可演进的 Skill Graph。重点将技能纳入图结构建模: 一方面将技能与企业实体、业务规则、合规约束建立关联; 另一方面显式刻画技能之间的依赖关系、互斥关系与替代关系, 形成可组合的技能工作流结构。针对多 Agent 知识的冗余与冲突, 技能图谱通过本体映射、语义融合与冲突消解完成统一治理, 并为每次新增/替换/降权更新绑定来源轨迹与证据链, 实现知识与技能的版本化管理。

在运行时, 演进后的技能图谱以“约束感知检索+可执行编排”的方式注入规划与执行: Agent 接收任务需求后, 结合任务目标、上下文权限、资源预算与规则约束, 在图谱上检索候选技能集

合及其依赖路径；随后将候选技能按依赖/互斥/替代关系编译为可执行的工作流，选择期望效用最大的方案执行。

最后，系统将每次任务执行形成的过程与结果指标回流作为知识自演进的反馈信号：一方面用于技能置信度与适用边界的增量更新，另一方面用于触发知识演进(新增技能提案、技能修订、关系重构等)。面向业务规则变化与能力漂移，建立反馈优化的闭环机制，实现知识与策略的持续优化，保证系统长期运行的稳态性能与适应业务变化的敏捷性。

在上述方案中，各关键模块分工明确、协同运作：

- 1) 轨迹解析与统一表示：完成异构轨迹的标准化、语义对齐与证据化封装，为后续技能提炼与治理提供高质量输入；
- 2) 技能化知识加工器：从轨迹中抽象技能并进行可信评估，实现隐性经验的模块化沉淀；
- 3) 技能图谱构建与演进：将技能与业务知识、合规约束及技能依赖关系统一建模，完成多源融合和冲突消解；
- 4) 知识增强决策优化：通过技能图谱检索增强优化 Agent 工作流编排与执行，并以在线反馈驱动持续演进；
- 5) 反馈与持续演进：驱动知识增量更新与回归验证，形成可持续优化的闭环强化机制。

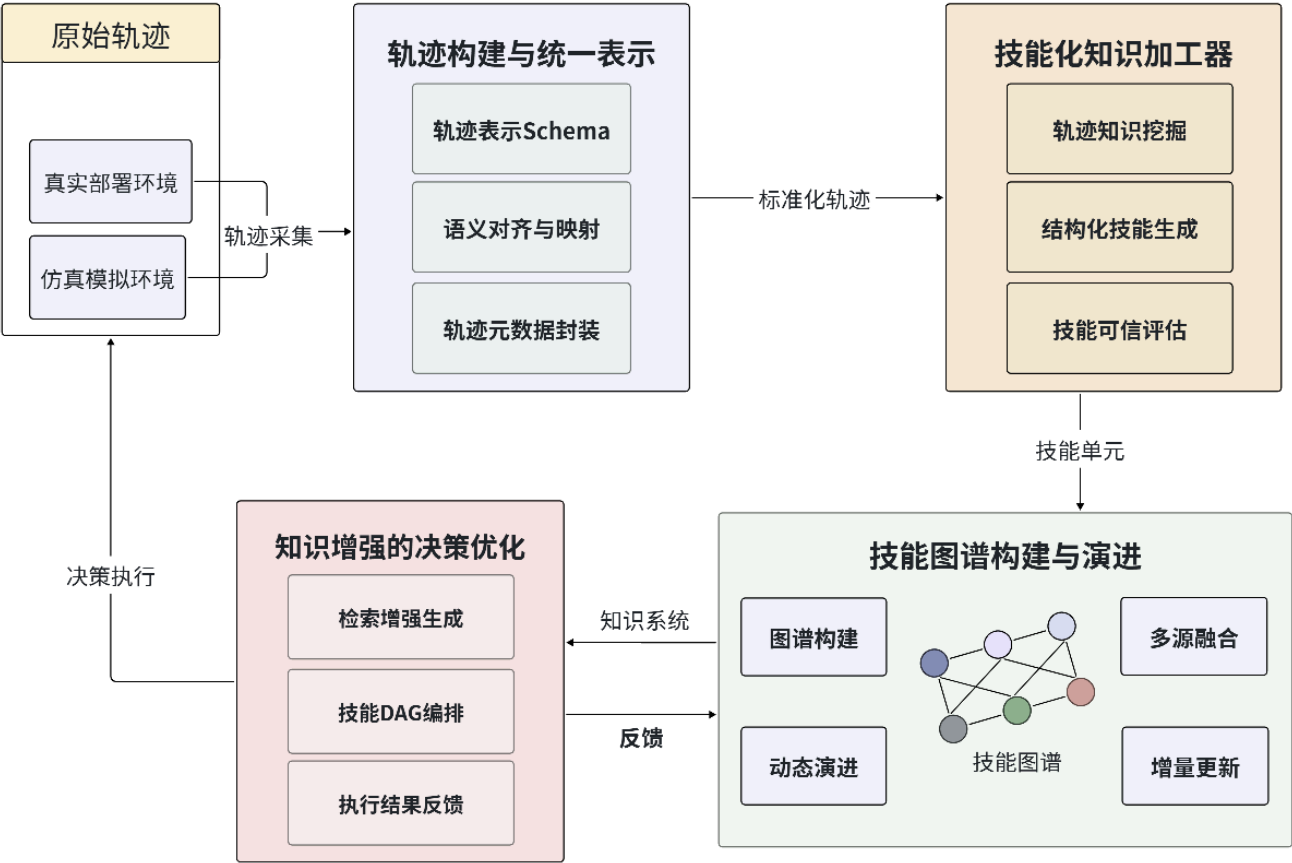


图 2：项目总体技术框架图

1.2.2 具体技术路线

为实现上述方案，本项目拟用 openJiuwen Agent 框架，按以下技术路线依次推进：

1. 轨迹数据构建与统一表示

核心任务是构建覆盖多业务场景(如客服工单、财务报销、IT 运维)的轨迹采集与回放管道，形成一个高质量数据底座。一方面，从真实部署环境采集多源轨迹(对话、计划、动作、工具/API 调用、观测反馈、异常与结果)；另一方面，在仿真环境中注入可控扰动(故障、权限变更、政策变更、数据漂移)合成难例轨迹，增强对例外知识与恢复策略的覆盖。随后，设计统一轨迹表示 Schema，将异构日志解析为标准化时序对象：

$$\tau = (s_t, p_t, a_t, o_t, r_t)_{t=1}^T,$$

其中 s_t 表示任务/环境状态, p_t 表示阶段性计划或子目标, a_t 表示动作(含工具调用及参数), o_t

表示观测反馈(含异常码、指标变化), r_t 表示结果或奖励信号(成功/失败、成本、时延等)。在此基础上完成跨系统字段与术语的语义对齐与映射, 并为每段轨迹绑定证据指针与审计元数据(来源、时间戳、权限、脱敏策略), 形成可计算、可链接、可追溯的 Agent 轨迹数据底座。

2. 技能化知识加工器

核心任务是将轨迹经验转化为可复用、可组合、可审计的技能单元, 而非停留在文本总结。首先进行轨迹切分与模式挖掘: 识别关键事件与状态转移边界, 抽取稳定的“条件—步骤—工具链—结果”结构, 并将其归纳为技能候选集合 $S = s_i$ 。每个技能以结构化规格表示:

$$s_i = \langle pre_i, io_i, tools_i, steps_i, stop_i, rollback_i, evidence_i \rangle,$$

分别对应前置条件、输入输出、依赖工具、步骤模板、终止条件、失败回退策略与证据链。其次, 引入成功/失败对照学习与回放重演验证, 评估技能的可迁移性与适用边界, 设计技能置信度分数, 对高置信技能进入可复用技能库并提交图谱治理, 从源头降低碎片经验复用导致的错误传播。

3. 技能图谱构建、融合与自演进

核心任务是构建可治理、可组合的技能知识结构, 并实现持续演进与版本可追溯。项目构建 Skill Graph:

$$SG = (V, E),$$

$$V = V_{skill}, V_{tool}, V_{biz}, V_{policy}, V_{failure}$$

$$E = depends_on, composed_of, alternative_to, mutual_excl, uses_tool, etc.$$

其中技能节点与业务实体、合规规则、失败模式建立显式关联, 技能之间以依赖/互斥/替代等关系刻画 workflow 结构。针对多源、多 Agent 的冗余与冲突, 采用本体映射与语义融合实现对齐, 并通过冲突消解策略保持一致性。进一步设计自演进机制: 当新增轨迹到达或关键指标漂移时触发增量更新

$$SG_{t+1} = Update(SG_t, \Delta T_t),$$

更新操作包含新增技能、替换/合并技能、降权淘汰、补充例外分支等, 并对每次更新绑定来源轨迹与证据链, 形成版本化演进记录, 确保知识结构可追溯、可审计。

4. 知识增强的决策优化

核心任务是把演进后的技能图谱转化为可直接驱动决策与执行的能力, 形成可度量、可持续优化的闭环。运行时, Agent 接收任务需求后, 1) 首先在技能图谱上进行约束感知的技能检索: 结合任务目标、上下文权限、合规规则与资源预算, 检索候选技能集合及其依赖路径; 2) 随后进入技能编排与 workflow 编译阶段, 将候选技能按依赖/互斥/替代关系编译为可执行的 Skill DAG, 并进行可执行性检查, 如前置条件满足性、工具可用性、权限合规性等; 3) 执行过程中依据图谱中的失败模式与 rollback 策略进行异常检测与自动恢复, 并对关键决策点输出可解释依据, 提升端到端任务成功率与运行稳定性。

5. 反馈驱动的持续演进

核心任务是将 Agent 线上运行产生的过程轨迹与结果指标转化为可控的演进信号, 形成“监测—诊断—提案—验证—发布—回滚”的闭环机制: 1) 系统持续采集任务、步骤与技能三个层面的反馈信号, 包括成功率、一次通过率、异常恢复率、成本时延、人工介入率与关键异常分布, 并将指标变化与对应的执行路径、技能节点及证据轨迹片段建立绑定关系; 2) 当检测到业务规则变化、工具接口变化或指标漂移时, 自动完成根因定位, 识别技能规格过时、适用边界漂移、替代路径或例外分支缺失、以及编排策略不当等问题, 生成结构化的知识演进提案, 覆盖新增技能、技能修订、关系重构与低效高风险技能降权淘汰; 3) 系统依据持续反馈对技能置信度与适用边界进行增量更新, 实现知识与策略的持续优化, 保证长期稳态性能与对业务变化的敏捷适应。

1.2.3 研究目标

围绕以上技术方案与路线，本项目预期实现以下 4 个目标：

- (1) **Agent 知识演进框架与工具链**：基于 openJiuwen 实现覆盖“轨迹统一表示→技能化知识加工→技能图谱 治理与版本演进→知识增强决策接口”的端到端框架，形成可插拔、易集成的工程化工具链并开源发布。
- (2) **开源数据集与评测基准**：面向典型场景构建轨迹级 Agent 能力评测数据集，建立评测协议与指标体系，用于客观衡量知识提炼质量与知识演进效果，并发布数据、基线方法与可复现评测脚本。
- (3) **知识强化效果的可量化提升**：在公开基准(如 AgentBench, ALFWorld, WebShop)与自建复杂任务轨迹基准上验证方法有效性，并在复杂规划、跨任务迁移等能力上相较 SOTA 提升 5%以上，形成完整的对比实验、消融分析与评测报告。
- (4) **面向真实场景完成试点部署与闭环验证**：选择不少于 2 类真实业务或科研场景开展试点，完成从轨迹采集、知识提炼、图谱治理到上线决策的端到端贯通，在真实环境的权限、合规与审计约束下实现稳定运行。

项目工作任务

工作描述	交付件	
完成项目调研工作和总体方案设计	技术调研报告 1 份	1. 需涵盖企业级 Agent 知识强化相关的前沿调研与技术洞察，覆盖 RAG、Agent Memory、轨迹挖掘/知识演进等方向； 2. 对现有方法优劣势、适用边界做定性对比分析，并形成可落地的技术选型结论。
	总体技术路线文档 1 份	总体技术路线需包含整体技术框架、算法详细设计，包含“轨迹统一表示、技能化知识加工、SKG 治理与版本演进、知识增强决策”完整链路
完成核心系统开发，包括轨迹统一表示、技能化知识加工、SKG 治理与版本演进、知识增强决策；提供运行	可执行源代码文件	1. 代码包含轨迹表示 Schema，环境部署，并能实现 AI4Science 场景下闭环：任务输入→编排→执行→反馈→知识更新。 2. 源代码完整且可执行； 3. 在公开数据集 AgentBench 上，通过知识强化 Agent 自演进，相比现有 SOTA 方

Demo		法，整体任务成功率提升 5%，F1 得分提高 5%，平均交互轮数减少 10%； 4. 在 openJiuwen 代码仓开源
	Demo 展示	基于 openJiuwen 平台搭建，分别针对 AI4Science 场景和模拟工业/金融场景构建 Demo，并提供可运行 Demo 和说明文档。
优化决策增强方法，构建评测数据集，性能评测与优化；真实场景验证	智能体决策增强方法源代码 1 份及技术文档 1 份	1. 源代码完整且可执行；文档内容完整、质量达标，通过华为验收团队的评审验收； 2. 在公开数据集 AgentBench 上，我们提出的方法相比现有 SOTA 方法，整体任务成功率提升 10%，F1 得分提高 10%，平均交互轮数减少 20%； 3. 在 openJiuwen 代码仓开源。
	轨迹级 Agent 能力评测数据集 1 个	1. 构建一个面向包含模拟工业场景、金融场景、AI4Science 场景的轨迹级 Agent 能力评测数据集，配套评测协议与指标体系。 2. 在 openJiuwen 代码仓开源。
	测试结果与评测报告 1 份	1. 提交测试结果与评测报告，包含指标、基线对比与复现步骤； 2. 完成科研场景的系统部署和测试。

1. 参考文献

[1] Zirui Guo et al., LightRAG: Simple and Fast Retrieval-Augmented Generation, EMNLP Findings, 2025.

[2] Haoyu Han et al., Retrieval-Augmented Generation with Graphs, arXiv:2404:1130, 2024.

[3] Patrick Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS, 2020

[4] Prateek Chhikara et al., Mem0: Building Production-Ready AI Agents with Scalable

Long-Term Memory, arXiv:2509.19413, 2025.

[5] Wujiang Xu et al., A-MEM Agentic Memory for LLM Agents, NeurIPS, 2025.

[6] Zhiyu Li et al., MemOS: A Memory OS for AI System, arXiv:2507.03724, 2025.

[7] Yu Wang et al., Mem- α : Learning Memory Construction via Reinforcement Learning, arXiv:2509.25911, 2025.

[8] Darshan Deshpande et al., TRAIL: Trace Reasoning and Agentic Issue Localization, arXiv:2505.08638, 2025.

[9] Siru Ouyang et al., ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory, arXiv:2509.25140, 2025

[10] Zhicheng Cai et al., FLEX: Continuous Agent Evolution via Forward Learning from Experience, arXiv:2511.06449, 2025